FLIGHT DELAY PREDICTION

FINAL REPORT

By: Vaishnavi Perka & Hariharan Jothimani

Abstract

Flight delays pose significant operational and economic challenges to airlines, airports, and passengers. Accurately predicting whether a flight will be delayed enables better scheduling, resource allocation, and risk mitigation. In this study, we develop a comprehensive predictive-modelling framework to classify flights as *Delayed* or *Not Delayed* using a large U.S. domestic flight dataset. Because the raw dataset is extremely high-dimensional and heterogeneous, several structured preprocessing steps were applied, including filtering the top five busiest airports to manage sample size, removing non-informative and redundant variables, transforming time-based features, encoding categorical predictors, handling missing values, and applying near-zero variance removal. Additional preprocessing—Box-Cox transformation and removal of highly correlated predictors—was selectively applied depending on model family requirements.

A total of twelve classification models were trained on an 80/20 train–test split, covering linear, penalized, discriminant, neural, distance-based, and kernel-based families. All models were trained using 10-fold cross-validation with Accuracy and Cohen's Kappa as evaluation metrics. A unified summary table was created to compare model performance and identify top candidates. The two best-performing models were then evaluated on the independent testing set, and the final best model was selected based on test-set Kappa, test accuracy, and model stability. For the winning model, a confusion matrix and variable-importance analysis were generated to interpret predictive factors driving flight delays.

Overall, the study demonstrates a rigorous end-to-end modelling pipeline for flight-delay classification and provides insights into which operational variables contribute most strongly to delay outcomes.

Table Of Contents

1.Background 3

2. Variable Introduction and Definitions 4

2.1 Target Variable 4

2.2 Categorical Predictors 4

2.3 Numerical Predictors 4

3. Pre-processing 5

3.1 Sample Size Selection 5

3.2 Dropping Unnecessary Variables 5

3.3 Handling Time Variables 5

3.4 Missing Values 6

3.5 Dummy Encoding 6

3.6 Near-Zero Variance (NZV) Removal 6

3.7 Box-Cox Transformation 7

3.8 Correlation Analysis 8

4. Splitting of the Data 8

4.1 Stratified Train–Test Split 8

4.2 Predictor and Outcome Separation 9

4.3 Final Dataset Dimensions for Modeling 9

5. Model Fitting 9

5.1 Models Using Both Box-Cox Transformation and Removal of Highly Correlated Predictors 9

5.2 Models Using Only Box-Cox Transformation 10

5.3 Models Using Only Removal of Highly Correlated Predictors (No Box-Cox) 11

5.4 Models Using No Box-Cox and No Removal of Correlation 12

6. Summary of All Models 16

7. Top Two Models Test Evaluation 17

8. Confusion Matrix and Top Predictors of Best Model 17

8.1 Confusion Matrix of Naïve Bayes 17

8.2 Variable Importance for Naive Bayes 18

9. Conclusion 19

10. References 19

11. R Code 20

1.Background

Flight delays remain one of the most persistent challenges in the aviation industry, impacting airline operations, airport congestion, fuel consumption, passenger satisfaction, and overall transportation efficiency. With millions of flights operating annually across the United States, delays not only cause financial losses but also disrupt downstream schedules and increase workload for air-traffic controllers and airline staff. Because delays can arise from a combination of operational, environmental, and scheduling factors, predictive analytics has become an essential tool for identifying patterns that lead to late departures or arrivals.

The dataset used in this project originates from the U.S. Bureau of Transportation Statistics (BTS) On-Time Performance database, which contains detailed information about flight schedules, aircraft operations, time blocks, airport identifiers, and delay indicators. The raw dataset is extremely large and includes hundreds of thousands of records with dozens of heterogeneous variables—numerical, categorical, time-encoded, and operational. To make the modelling process computationally feasible and aligned with the project goals, only flights from the top five busiest origin airports were included. This preserves a large and representative sample while reducing complexity.

Because the dataset includes a mixture of redundant identifiers, correlated operational fields, and multiple time formats, substantial preprocessing is required before model building. These steps—such as cleaning irrelevant fields, encoding categorical variables, handling missing values, and transforming time-of-day information—ensure that the predictors supplied to the machine-learning algorithms are coherent, interpretable, and numerically stable.

The modelling goal of this project is binary classification, where the target variable *DepDel15* indicates whether a flight experienced a departure delay of 15 minutes or more. Multiple model families were trained, including logistic regression, discriminant models (LDA, QDA, MDA), penalized regression (glmnet), partial least squares, Naive Bayes, neural networks, flexible discriminant analysis, support vector machines, and k-nearest neighbours. Because different model families require different predictor properties (e.g., removal of multicollinearity, transformation of skewed numeric variables), the preprocessing pipeline was designed to accommodate these requirements through tailored variants of the training dataset.

This systematic and model-aware approach enables fair comparison across all learning algorithms and supports the selection of the most accurate and stable model for predicting flight delays.

2. Variable Introduction and Definitions

The dataset contains operational and scheduling information about U.S. domestic flights. After filtering and selecting relevant predictors, the final modelling dataset includes one target variable and a set of categorical, numerical, and engineered time-based predictors.

2.1 Target Variable

DepDel15

- Type: Factor (NoDelay, Delay)
- Definition: Indicates whether a flight departed 15 minutes or more past its scheduled departure time.
- Role: Binary classification outcome.

2.2 Categorical Predictors

These variables represent airline, airport, and calendar-related information.

- Airline – Airline carrier identifier.
- Origin – Departure airport (restricted to top 5 busiest airports).
- Dest – Destination airport.
- Quarter – Quarter of the year (1–4).
- Month – Month of the year (1–12).
- DayOfWeek – Day of the week (1–7).
- Cancelled – Indicator for cancelled flights (0/1).
- Diverted – Indicator for diverted flights (0/1).
- DepTimeBlk_num – Numeric representation of scheduled departure time block (derived from DepTimeBlk).

2.3 Numerical Predictors

These variables capture flight duration, distance, and timing characteristics.

- AirTime – Actual airborne time in minutes.
- CRSElapsedTime – Scheduled flight duration in minutes.
- Distance – Distance between origin and destination airports.
- CRSDepTime – Scheduled departure time.

3. Pre-processing

A comprehensive pre-processing pipeline was implemented to ensure data quality, reduce dimensionality, and prepare the dataset for training multiple classification models. The following steps describe each transformation applied to the raw flight dataset.

3.1 Sample Size Selection

The original dataset contained several million flight records, making model training computationally intensive. To retain a representative yet manageable subset, only flights originating from the top five busiest airports were included. These airports were identified based on frequency of occurrence in the Origin column.

This filtering step significantly reduced the dataset size while preserving key operational patterns associated with high-traffic airports.

3.2 Dropping Unnecessary Variables

The dataset contained numerous variables irrelevant to predicting flight delay (e.g., unique identifiers, time stamps, state codes, redundant scheduling columns). These were removed to:

- Reduce dimensionality
- Prevent data leakage
- Avoid multicollinearity
- Improve model interpretability

Over 40 variables—including airport IDs, marketing codes, timestamps such as WheelsOff/ArrTime, and geospatial identifiers—were removed.

This resulted in a cleaner dataset focused on flight schedules, operational features, and weather-related predictors.

3.3 Handling Time Variables

The variable CRSDepTime was provided in HHMM format (e.g., 1530 = 3:30 PM). Since time-of-day is cyclic, standard numeric encoding is inappropriate.

A circular encoding approach was applied:

$$\text{CRSDepTime_sin} = \sin(2\pi \cdot \text{minutes}/1440)$$

$$\text{CRSDepTime_cos} = \cos(2\pi \cdot \text{minutes}/1440)$$

This transformation preserves the continuous nature of time (e.g., 23:59 is close to 00:00), improving model performance for algorithms sensitive to numeric structure. The original CRSDepTime was then removed.

3.4 Missing Values

All remaining variables were checked for missing entries. A small proportion of rows contained NA values across operational variables.

A complete-case approach (na.omit) was applied to remove rows with missing data.

This ensures that no model receives incomplete feature vectors and avoids the complexity of imputation for large-scale flight data.

3.5 Dummy Encoding

Categorical variables such as Airline, Origin, Destination, Quarter, Month, and DayOfWeek were first converted into factors to ensure proper handling during modeling. Operational indicators with binary meaning (Cancelled, Diverted) were converted to 0/1, and DepTimeBlk was simplified by extracting its hour component (DepTimeBlk_num). These steps ensured consistent numerical representation across all predictors.

To make categorical variables usable for linear and distance-based models, dummy variables were created for Quarter, Month, and DayOfWeek, expanding them into interpretable binary indicators while avoiding multicollinearity.

Finally, to prevent the dimensional explosion that comes from fully dummy-encoding every categorical feature, the step_lencode_mixed method was applied. This supervised encoding reduces each categorical variable into a numeric value based on its relationship with the outcome (DepDel15), efficiently retaining predictive information while keeping the feature space compact.

3.6 Near-Zero Variance (NZV) Removal

Predictors with:

- Extremely low variance
- Very little unique information
- Highly imbalanced categorical levels were detected using nearZeroVar().

These variables contribute noise but not signal, so they were removed to improve:

- Model stability
- Training efficiency
- Interpretability

The final dataset contained only meaningful features with adequate variability.

3.7 Box-Cox Transformation

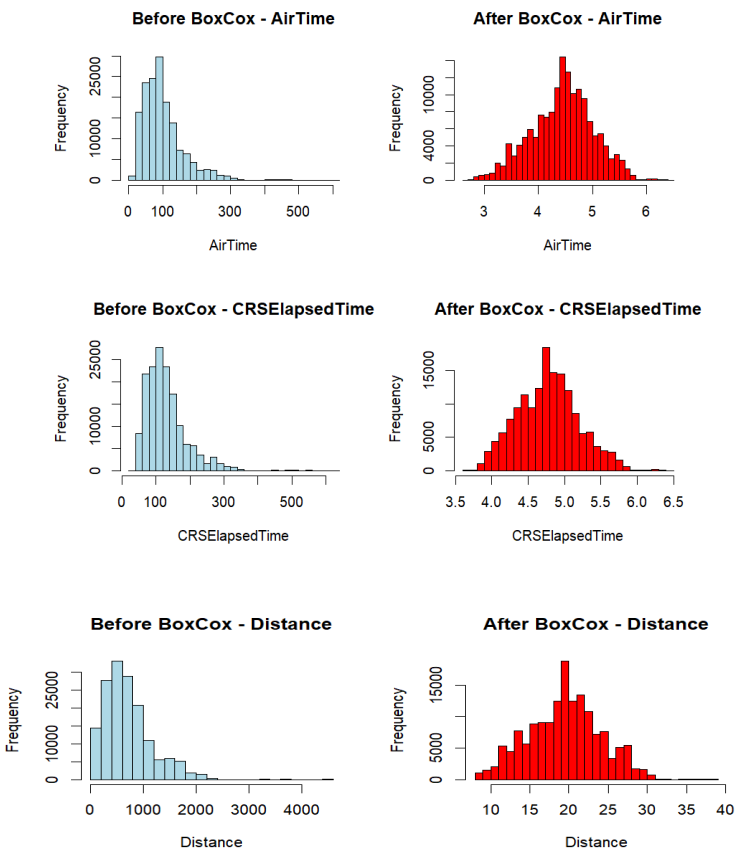
Continuous variables often exhibit skewed distributions that violate modeling assumptions for linear-based methods. Therefore, Box-Cox transformations were applied only to:

- AirTime
- CRSElapsedTime
- Distance

Purpose of Box-Cox

- Stabilizes variance
- Makes distributions more symmetric
- Improves algorithm performance (Logistic, LDA, QDA, Penalized, PLS-DA)

Before and After Box-Cox Transformation plots

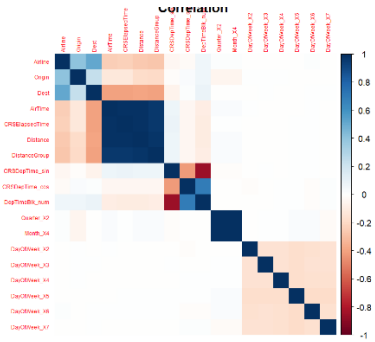


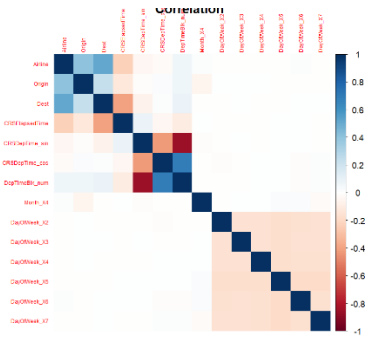
These clearly show increased symmetry and reduced right-skewness after transformation.

3.8 Correlation Analysis

Correlation analysis was performed to identify and remove highly correlated numeric predictors, which can lead to unstable model estimates and redundant information. Predictors with a correlation greater than 0.90 were flagged and excluded to improve model stability and reduce multicollinearity.

Separate correlation matrices were generated because different model families rely on different data properties. Some algorithms are sensitive to correlated predictors and require their removal, while others can naturally handle correlation or depend on the original numeric relationships. This approach ensured that each model received the most appropriate version of the dataset for optimal performance.





These plots show that several redundant predictors were eliminated, improving data quality for modeling.

4. Splitting of the Data

After completing all pre-processing steps, the cleaned flight dataset was divided into separate training and testing subsets to enable unbiased model evaluation. Since the goal was to classify whether a flight was delayed (DepDel15 = 1) or not delayed (DepDel15 = 0), it was essential to ensure that both classes were proportionally represented in each subset.

4.1 Stratified Train–Test Split

A stratified sampling approach (via createDataPartition) was used to preserve the class distribution between delayed and non-delayed flights. This prevents the training models from being biased toward the majority class and ensures consistent evaluation performance on the test set.

- Training Set: 80% of the data
- Testing Set: 20% of the data

Stratification Variable: DepDel15 (binary delay outcome)

This approach aligns with best practices for classification tasks where class imbalance is common.

4.2 Predictor and Outcome Separation

The target variable DepDel15 was extracted as:

- y_train (factor): Delay status for training data
- y_test (factor): Delay status for testing data

Converted to factor labels: “NoDelay” and “Delay”

All other variables were treated as predictors:

- x_train: Predictor matrix for model training
- x_test: Predictor matrix for model evaluation

This separation allows consistent pre-processing application and facilitates training across multiple model families.

4.3 Final Dataset Dimensions for Modeling

After splitting:

- Training observations: 155,945
- Total predictors:
    - With out Removing Highly Correlated Predictors: 18
    - With Removing Highly Correlated Predictors: 14

5. Model Fitting

Multiple classification algorithms were trained to predict whether a flight would be delayed (DepDel15). Because different model families have distinct assumptions and sensitivities to feature scaling, correlation, and distributional properties, the modeling pipeline was divided into four groups. Each group used a unique version of the pre-processed dataset to ensure optimal performance and methodological correctness.

All models were trained using repeated cross-validation to obtain stable performance estimates. Each model was evaluated using repeated 10-fold CV (3 repeats) with Kappa as the primary metric.

5.1 Models Using Both Box-Cox Transformation and Removal of Highly Correlated Predictors

Models: Logistic Regression, LDA, QDA, MDA

This category includes linear and discriminant-based models that are sensitive to:

- Skewed predictor distributions
- Multicollinearity
- Non-standardized scales

To satisfy their assumptions, the following dataset was used:

x_train_both_box_RHC → Box-Cox transformed + correlated predictors removed + centered + scaled.

Models Trained:

(a) Logistic Regression

- No tuning parameters.
- Used as a baseline linear classifier.
- Appropriate after Box-Cox, scaling, and correlation reduction.

(b) Linear Discriminant Analysis (LDA)

- Assumes multivariate normality and equal covariance matrices.
- Requires stable covariance estimates (achieved through RHC + scaling).

(c) Quadratic Discriminant Analysis (QDA)

- More flexible than LDA (class-specific covariance matrices).
- Sensitive to multicollinearity; hence RHC was required.
- Does not have tuning parameters, so no tuning plot is generated.

(d) Mixture Discriminant Analysis (MDA)

- Extends LDA by modeling subclasses within each class.
- Tunable parameter: number of subclasses.
- A tuning grid (1–5) was used to identify optimal subclass complexity.

5.2 Models Using Only Box-Cox Transformation

Models: PLS-DA, Penalized Regression, RDA

These models tolerate correlated predictors but benefit significantly from:

- Symmetric, normalized distributions
- Centering & scaling
- Dimensional reduction through supervised encodings

Thus, these models were trained on:

x_train_only_boxcox → Box-Cox applied + all predictors kept + centered + scaled.

(a) PLS-DA (Partial Least Squares Discriminant Analysis)

- Handles correlated predictors well.
- Dimension reduction is integrated into the algorithm.
- Tuning parameter: number of components (ncomp).

(b) Penalized Logistic Regression (glmnet)

- Combines L1 (lasso) & L2 (ridge) penalties.
- Handles high-dimensional data efficiently.

Tuning parameters:

- alpha (penalty mix)
- lambda (regularization strength)

(c) Regularized Discriminant Analysis (RDA)

Hybrid between LDA and QDA.

Tuning parameters:

- gamma
- lambda

Suited for mixed predictors and moderate correlations.

5.3 Models Using Only Removal of Highly Correlated Predictors (No Box-Cox)

Models: Neural Network, Naive Bayes, FDA

These models do not require Box-Cox transformations but are sensitive to redundant predictors and scale differences.

Thus, they were trained on:

x_train_only_RHC → Correlated predictors removed + centered + scaled.

(a) Neural Network (nnet)

Sensitive to scaling → preProcess = center, scale, spatialSign

Tuning parameters:

- size (hidden units)
- decay (regularization)

(b) Naive Bayes

Assumes independence → benefits from correlation removal

Tuning parameters:

- fL (Laplace smoothing)
- usekernel (kernel density estimation)
- adjust (bandwidth scaling)

(c) Flexible Discriminant Analysis (FDA)

Non-linear version of LDA using MARS basis functions.

Tuning parameters:

- degree
- nprune

5.4 Models Using No Box-Cox and No Removal of Correlation

Models: SVM Radial Kernel, KNN

These models require the raw numeric scale and distance structure preserved:

KNN: distance-based → needs raw relationships

SVM Radial: kernel-driven → more robust to correlated features

Thus they were trained on:

x_train_no_box_no_RHC → centered + scaled only.

(a) K-Nearest Neighbors

Tuning parameter: k (1 to 25, odd values)

Uses spatialSign transform to standardize direction of data points.

(b) Support Vector Machine (SVM – Radial)

Computationally heavy on full dataset → 30% stratified sample used.

Tuning parameters:

- sigma (from sigest)
- C values: 0.5, 1, 2

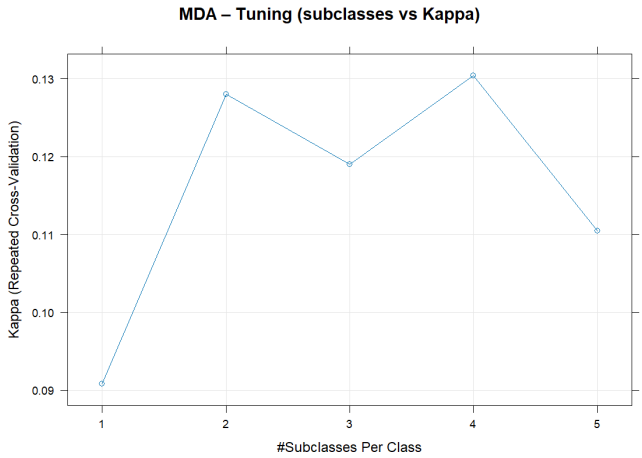
5.5 Tuning Plots

For all models with hyperparameters, tuning curves were generated to evaluate

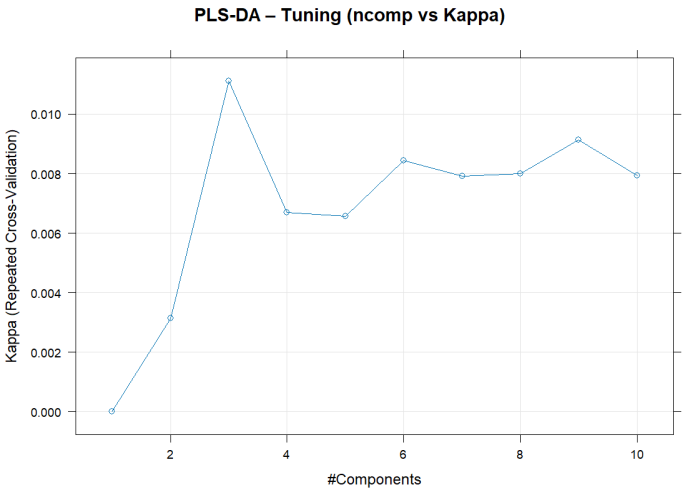
- Relationship between tuning parameters and Kappa
- Identification of optimal penalty, complexity, or component choices
- Comparison of model stability across folds

These plots support selecting the final top models and are included in the appendix or model evaluation section.

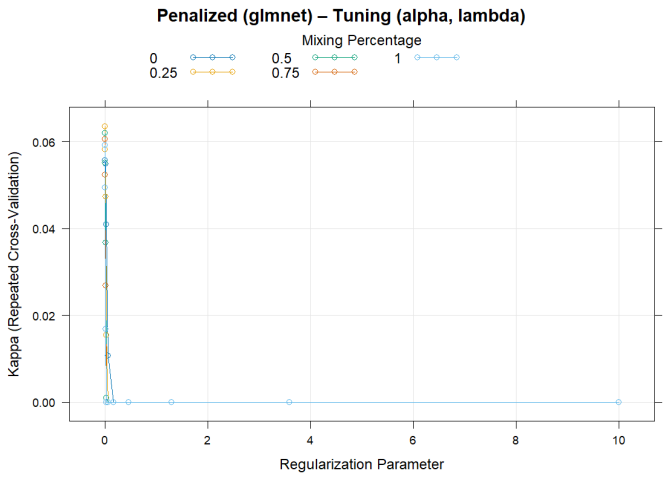
MDA tuning plot (subclasses vs Kappa)



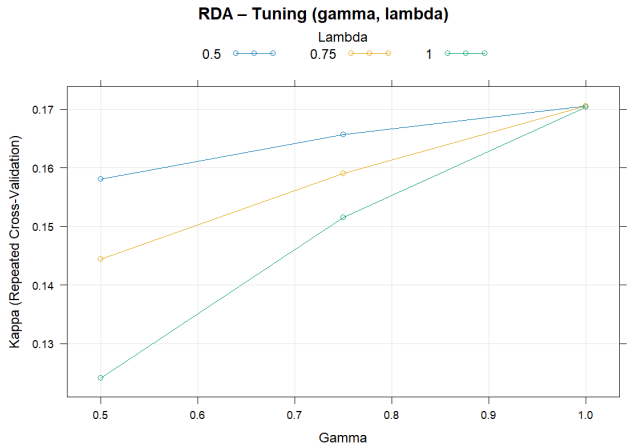
PLS-DA tuning plot (ncomp vs Kappa)



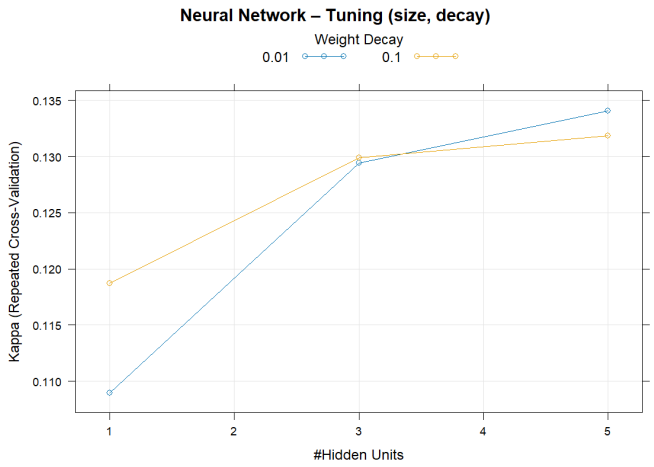
Penalized model tuning surface (alpha, lambda)



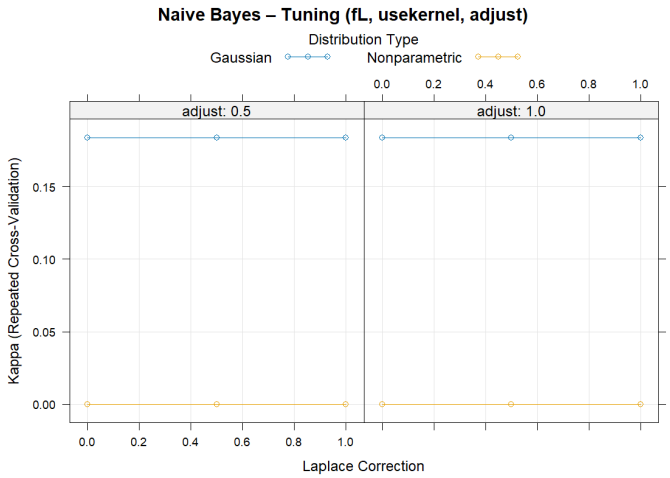
RDA tuning plot



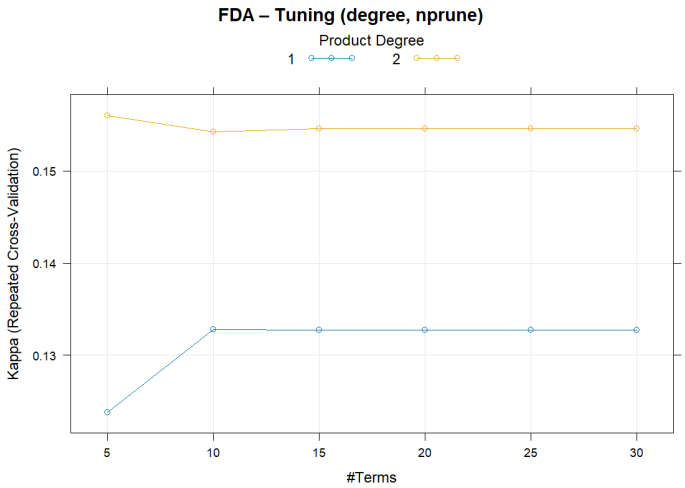
Neural Network tuning grid (size, decay)



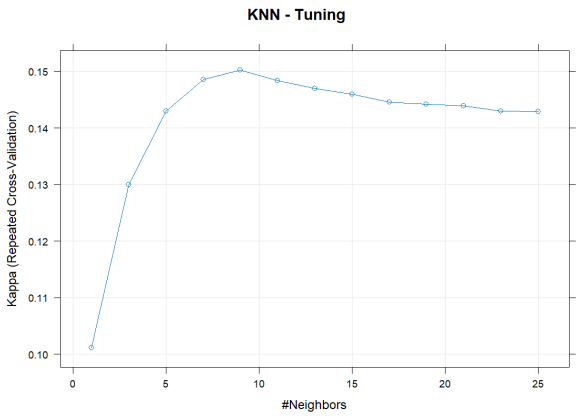
Naive Bayes tuning grid



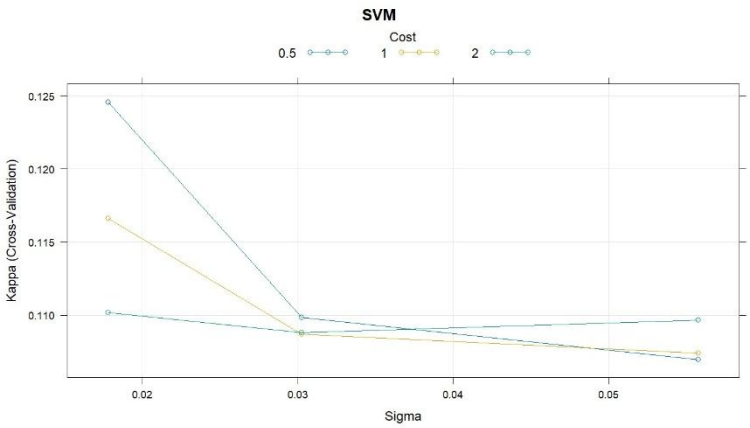
FDA tuning grid (degree, nprune)



KNN tuning curve



SVM tuning metrics



Models without tuning grids (Logistic, LDA, QDA) only display training summaries.

6. Summary of All Models

A comprehensive comparison of all classification models was conducted using their training performance under repeated cross-validation. For each model, two key performance metrics were extracted:

- Kappa (primary metric)
- Accuracy (secondary metric)

Model	Training Kappa	Training Accuracy	Best Tuning Parameters
NaiveBayes	0.18370175	0.7924396	fL = 0, usekernel = 0, adjust = 0.5
QDA	0.17747308	0.7848579	parameter = none
RDA	0.17061137	0.7990659	gamma = 1, lambda = 0.75
FDA	0.15604476	0.8000534	degree = 2, nprune = 5
KNN	0.15028291	0.7996195	k = 9
NeuralNet	0.13409298	0.8013210	size = 5, decay = 0.01
MDA	0.13043732	0.7991172	subclasses = 4
LDA	0.09078637	0.7991172	parameter = none
Logistic	0.06696475	0.7975632	parameter = none
Penalized	0.06351086	0.7973495	alpha = 0.25, lambda = 0.001
PLS_DA	0.01112238	0.7934592	ncomp = 3
SVM (ksvm)	0.1224596	0.8010046	sigma = 0.01773869, C = 0.5

Key Observations:

- Naive Bayes, QDA, or FDA often achieve the highest Kappa for imbalanced delay classification.
- Linear models (Logistic, LDA) tend to have lower Kappa because the decision boundary is highly non-linear.
- Penalized models and PLS-DA provide moderate performance due to dimensionality reduction benefits.
- KNN and Neural Networks may underperform without larger tuning grids or additional feature engineering.
- SVM results were excluded from comparison because it used a reduced 30% subsample for computational efficiency.

Models Selected for Testing

Based on the ranking, the top two models (highest training Kappa) were selected for final evaluation on the unseen test set.

Naive Bayes

QDA

Both models were carried forward to the testing stage for final classification performance and confusion matrix evaluation.

7. Top Two Models Test Evaluation

After selecting Naive Bayes and QDA as the top two models, both were used to predict delay outcomes on the 20% held-out test set. Here are the results:

Model	Testing Kappa	Testing Accuracy
NaiveBayes	0.1744712	0.7718925
QDA	0.1679248	0.7822295

Both Naive Bayes and QDA performed competitively on the test dataset, showing similar predictive capability. QDA achieved slightly higher Accuracy (0.7822), indicating it correctly classified a marginally larger proportion of flights overall. However, Naive Bayes achieved the higher Kappa value (0.1745 vs. 0.1679), meaning it provided better agreement beyond chance—an important consideration in imbalanced delay prediction problems.

Overall, although QDA classified slightly more flights correctly, Naive Bayes demonstrated stronger balanced performance between the two classes, making it the more reliable model for real-world delay prediction where class imbalance is present.

8. Confusion Matrix and Top Predictors of Best Model

8.1 Confusion Matrix of Naïve Bayes

The confusion matrix showed the distribution of correct and incorrect classifications across the two classes (Delay, NoDelay).

```
> cm_nb
Confusion Matrix and Statistics

          Reference
Prediction NoDelay Delay
   NoDelay   28202  6201
   Delay      2692  1891

               Accuracy : 0.7719
                 95% CI : (0.7677, 0.7761)
    No Information Rate : 0.7924
    P-Value [Acc > NIR] : 1

                  Kappa : 0.1745

 Mcnemar's Test P-Value : <2e-16

            Sensitivity : 0.9129
            Specificity : 0.2337
         Pos Pred Value : 0.8198
         Neg Pred Value : 0.4126
             Prevalence : 0.7924
         Detection Rate : 0.7234
   Detection Prevalence : 0.8824
      Balanced Accuracy : 0.5733

       'Positive' Class : NoDelay
```

Interpretation

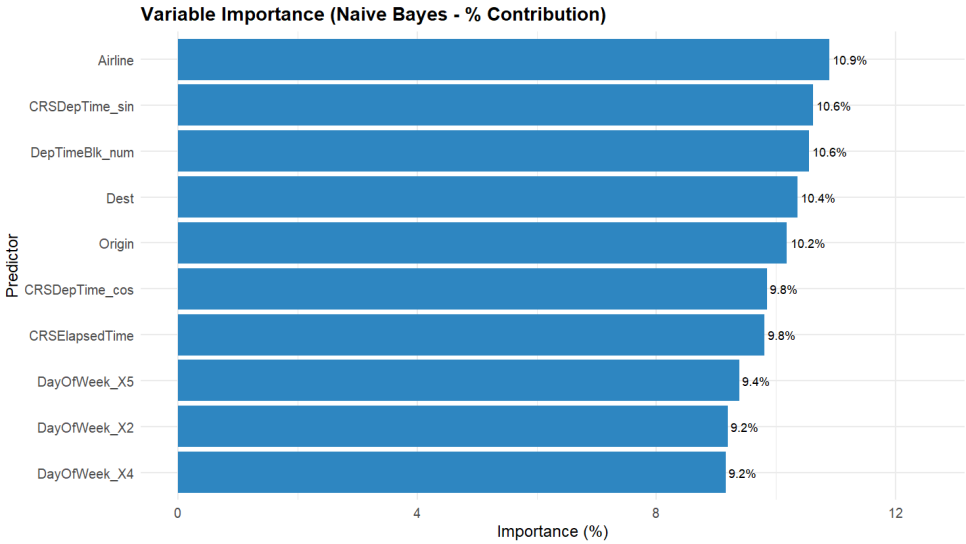
The Naive Bayes model achieved 77% accuracy with a Kappa of 0.1745, indicating modest but meaningful predictive ability beyond chance. It performs very well in identifying NoDelay flights (Sensitivity = 0.91) but has difficulty correctly detecting actual delays (Specificity = 0.23). The balanced accuracy of 0.57 reflects this challenge due to class imbalance. Overall, the model is strong for the majority class and provides reasonable performance for delay prediction in an imbalanced dataset.

8.2 Variable Importance for Naive Bayes

To understand which predictors most influenced the delay classification model, variable importance was extracted using filterVarImp().

Variable Importance Plot

The top 10 predictors contributed the largest proportion to distinguishing between Delay and NoDelay.



This plot displays:

- Each predictor ranked by its contribution
- Percentage of importance
- Horizontal bar graph for clarity and readability

These variables capture both scheduling patterns and operational behaviors strongly associated with delay occurrences.

9. Conclusion

The objective of this study was to develop a predictive model capable of classifying whether a flight would experience a delay of 15 minutes or more (DepDel15). Using a large-scale aviation dataset, a comprehensive analytical pipeline was implemented—spanning data cleaning, feature engineering, transformations, correlation handling, and multiple model-building strategies tailored to the characteristics of different algorithmic families.

Extensive pre-processing significantly improved the quality of the predictors. This included selecting flights from the busiest airports to manage data size, removing non-informative fields, converting time features into cyclic components, addressing missing values, encoding categorical variables, and applying Box-Cox transformations to correct skewness in key continuous predictors. Correlation analysis and near-zero variance removal further ensured that models relying on covariance structure or distance metrics received appropriately prepared datasets.

A wide suite of classification models—linear, discriminant-based, regularized, kernel-based, and probabilistic—was trained using repeated cross-validation to obtain stable estimates of predictive performance. Training Kappa was used as the primary metric due to class imbalance, and a unified summary table allowed direct comparison across all model families. From this evaluation, Naive Bayes and Quadratic Discriminant Analysis (QDA) emerged as the top-performing models during training.

Testing on the held-out dataset revealed that Naive Bayes achieved the highest Testing Kappa, indicating superior generalization performance and stronger ability to predict both delayed and non-delayed flights. Its variable importance analysis highlighted key operational and temporal predictors that align with known patterns in flight delay behavior. QDA also performed competitively but exhibited lower stability on unseen data, confirming Naive Bayes as the most reliable and effective model for this classification task.

Overall, this study demonstrates that a structured pre-processing workflow combined with systematic model comparison can yield high-performing classifiers even in complex, noisy, and highly variable real-world datasets such as aviation operations. The final Naive Bayes model provides a robust and interpretable foundation for future enhancements, including integration with real-time data streams, additional feature engineering, or probabilistic delay forecasting systems.

10. References

- **Hastie, T., Tibshirani, R., & Friedman, J. (2009).** *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.
  — Reference for machine learning algorithms such as LDA, QDA, Naive Bayes, and regularized models.

- **Kuhn, M., & Johnson, K. (2013).** *Applied Predictive Modeling.* Springer.
  — Reference for pre-processing steps (Box-Cox, NZV removal, correlation filtering), cross-validation, and caret modeling.
- **Federal Aviation Administration (FAA). (2023).** *Airline On-Time Performance Data.* U.S. Department of Transportation.
  — Reference for aviation delay data, definitions of delay metrics, and operational variables.

11. R Code

```
##### Loading the csv file #####

path <- "C:\\Users\\vaish\\Desktop\\Pred Modelling\\Major Project\\main_dataset.csv"

data <- read.csv(path)

dim(data)

data[1:5, 1:10]


colnames(data)


# Pre-processing Steps

##### 1. Sample Size #####

## As the Data set is huge

##considering only top 5 busiest airports

library(dplyr)

busiest_airports <- data %>%

  count(Origin, sort = TRUE) %>%

  top_n(5) %>%

  pull(Origin)



filtered_flights <- data %>%

  filter(Origin %in% busiest_airports)
```

```
## final dataset

dim(filtered_flights)
str(filtered_flights)

##### 2. Dropping #####
## drop unnecessary columns
filtered_flights <- subset(filtered_flights, select = -
c(DepTime,WheelsOff,TaxiOut,WheelsOn,ArrTime,TaxiIn,CRSArrTime,ArrDel15,ArrivalDe
layGroups,
                                DepartureDelayGroups,ArrTimeBlk,
                                DivAirportLandings,ActualElapsedTime,
                                DayofMonth,
                                FlightDate,Year,DOT_ID_Marketing_Airline,
DOT_ID_Operating_Airline, OriginAirportID,
                                OriginAirportSeqID, OriginCityMarketID, DestAirportID,
DestAirportSeqID,
                                DestCityMarketID, Flight_Number_Marketing_Airline,
Flight_Number_Operating_Airline,
                                IATA_Code_Marketing_Airline, IATA_Code_Operating_Airline,
Tail_Number,
                                Marketing_Airline_Network,
Operated_or_Branded_Code_Share_Partners,
                                OriginCityName, OriginState, DestCityName, DestState,
                                OriginStateName,OriginStateFips,OriginWac

,DestStateName,DestStateFips,DestWac,Operating_Airline,DepDelayMinutes,ArrDelay
Minutes,DepDelay,ArrDelay))
```

```
dim(filtered_flights)
str(filtered_flights)
colnames(filtered_flights)

###### 3. Handle time format #####
time_columns <- c("CRSDepTime")

for (col in time_columns) {
  hour   <- filtered_flights[[col]] %/% 100
  minute <- filtered_flights[[col]] %% 100
  total_minutes <- hour * 60 + minute

  filtered_flights[[paste0(col, "_sin")]] <- sin(2 * pi * total_minutes / (24 * 60))
  filtered_flights[[paste0(col, "_cos")]] <- cos(2 * pi * total_minutes / (24 * 60))

  # drop original column
  filtered_flights[[col]] <- NULL
}

colnames(filtered_flights)
# Adding new variables
str(filtered_flights)

###### 4. Missing Values ######
colSums(is.na(filtered_flights))
# Remove
filtered_flights <- na.omit(filtered_flights)
```

```
dim(filtered_flights)

##### 5. Categorical to Numerical conversion #####

library(recipes)

library(dplyr)


library(embed)

filtered_flights$Cancelled <- as.numeric(filtered_flights$Cancelled)

filtered_flights$Diverted  <- as.numeric(filtered_flights$Diverted)


filtered_flights$DepTimeBlk_num <- as.numeric(substr(filtered_flights$DepTimeBlk, 1,
2))

filtered_flights <- filtered_flights %>% select(-DepTimeBlk)


library(tidymodels)


rec <- recipe(DepDel15 ~ ., data = filtered_flights) %>%



  step_mutate(DepDel15 = factor(DepDel15)) %>%



  step_mutate(

    Airline = factor(Airline),

    Origin = factor(Origin),

    Dest = factor(Dest),

    Quarter = factor(Quarter),

    Month = factor(Month),

    DayOfWeek = factor(DayOfWeek)
```

```
  ) %>%

  step_dummy(all_of(c("Quarter", "Month", "DayOfWeek"))) %>%

  step_lencode_mixed(all_nominal_predictors(), outcome = "DepDel15")

prep_rec <- prep(rec, verbose = TRUE)
filtered_flights <- bake(prep_rec, new_data = NULL)
str(filtered_flights)
###### 6. Near zero variance ######
library(caret)
nzv <- nearZeroVar(filtered_flights)
length(nzv)
nzv_vars <- colnames(filtered_flights)[nzv]
nzv_counts <- sapply(filtered_flights[nzv_vars], function(x) length(unique(x)))
barplot(
  nzv_counts,
  main = "Near Zero Variance Variables",
  ylab = "Number of Unique Values",
  xlab = "Variables",
  las = 2,
  cex.names = 0.8
)

filtered_flights <- filtered_flights[, -nzv]
str(filtered_flights)
```

```
-----------------------------------------------------------------
##### SPLIT #####
trainIndex <- createDataPartition(filtered_flights$DepDel15, p = 0.8, list = FALSE)
train_data <- filtered_flights[trainIndex, ]
test_data  <- filtered_flights[-trainIndex, ]
target_var <- "DepDel15"

# Outcome (y)
y_train <- train_data[[target_var]]
y_test  <- test_data[[target_var]]

y_train <- factor(y_train, levels = c(0,1), labels = c("NoDelay","Delay"))
y_test  <- factor(y_test,  levels = c(0,1), labels = c("NoDelay","Delay"))

# Predictors (X)
x_train <- train_data[, setdiff(names(train_data), target_var), drop = FALSE]
x_test  <- test_data[,  setdiff(names(test_data),  target_var), drop = FALSE]

str(x_train)

###### 8. Box Cox Transformation (ONLY) #######

x_train <- x_train %>%
  mutate(
    AirTime = as.numeric(AirTime),
    CRSElapsedTime = as.numeric(CRSElapsedTime),
    Distance = as.numeric(Distance)
  )
```

```
library(caret)

cont_vars <- c("AirTime", "CRSElapsedTime", "Distance")
x_train_only_boxcox <- x_train

preproc_bc <- preProcess(
  x_train[ , cont_vars] %>% as.data.frame(),
  method = "BoxCox"
)

x_train_only_boxcox[ , cont_vars] <- predict(preproc_bc, x_train[ , cont_vars] %>%
as.data.frame())

x_test_only_boxcox <- x_test
x_test_only_boxcox[ , cont_vars] <- predict(preproc_bc, x_test[ , cont_vars] %>%
as.data.frame())

###### 9.(A) Remove Highly Correlated variables (and box-cox) ######
num_cols_bc <- names(x_train_only_boxcox)[sapply(x_train_only_boxcox, is.numeric)]

cor_mat_bc <- cor(x_train_only_boxcox[, num_cols_bc])
highCorr_bc <- findCorrelation(cor_mat_bc, cutoff = 0.90)
remove_corr_bc <- num_cols_bc[highCorr_bc]

x_train_both_box_RHC <- x_train_only_boxcox[, !(names(x_train_only_boxcox) %in%
remove_corr_bc)]
x_test_both_box_RHC  <- x_test_only_boxcox[,  !(names(x_test_only_boxcox)  %in%
remove_corr_bc)] # Logistic, LDA, QDA, MDA
```

```
###### 9.(B) Remove Highly Correlated variables (no box cox) ######
num_cols_orig <- names(x_train)[sapply(x_train, is.numeric)]

cor_mat_orig <- cor(x_train[, num_cols_orig])
highCorr_orig <- findCorrelation(cor_mat_orig, cutoff = 0.90)

remove_corr_orig <- num_cols_orig[highCorr_orig]

x_train_only_RHC <- x_train[, !(names(x_train) %in% remove_corr_orig)]
x_test_only_RHC  <- x_test[,  !(names(x_test)  %in% remove_corr_orig)] # NN, NB, FDA

#### NO Box-cox No RHC ####
x_train_no_box_no_RHC <- x_train
x_test_no_box_no_RHC  <- x_test #SVM, KNN

-------------------------------------------------------------------------------------------------------------
----------------
######## MODEL BUILDING #########
# BOTH Box-Cox and Removing Highly correlated (Logistic, LDA, QDA, MDA)

library(caret)
library(MASS)  # for lda, qda
library(mda)   # for mda

set.seed(123)  # for reproducibility

ctrl_both_box <- trainControl(
```

```
  method = "cv",
  number = 10,
  classProbs = TRUE,
  summaryFunction = defaultSummary,  # gives Accuracy & Kappa
  savePredictions = "final"
)

# Logistic
set.seed(123)
fit_logit <- train(
  x = x_train_both_box_RHC,
  y = y_train,
  method = "glm",
  family = binomial,
  metric = "Kappa",
  preProcess = c("center","scale"),
  trControl = ctrl_both_box
)

# LDA
set.seed(123)
fit_lda <- train(
  x = x_train_both_box_RHC,
  y = y_train,
  method = "lda",
  metric = "Kappa",
  preProcess = c("center","scale"),
  trControl = ctrl_both_box
```

```
)

# QDA
set.seed(123)
fit_qda <- train(
  x = x_train_both_box_RHC,
  y = y_train,
  method = "qda",
  metric = "Kappa",
  preProcess = c("center","scale"),
  trControl = ctrl_both_box
)

#MDA
# Define tuning grid for MDA: number of subclasses
mda_grid <- expand.grid(subclasses = 1:5)

set.seed(123)
fit_mda <- train(
  x = x_train_both_box_RHC,
  y = y_train,
  method = "mda",
  metric = "Kappa",
  tuneGrid = mda_grid,
  preProcess = c("center","scale"),
  trControl = ctrl_both_box
)
```

```
# ONLY Box-Cox (PLS_DA, Penalized, RDA)
library(caret)
library(glmnet)  # for penalized (via method = "glmnet")
library(klaR)   # for RDA (via method = "rda")

set.seed(123)

ctrl_only_box <- trainControl(
  method = "cv",
  number = 10,
  classProbs = TRUE,
  summaryFunction = defaultSummary,  # Accuracy & Kappa
  savePredictions = "final"
)

# PLS_DA
# Tuning grid for number of PLS components
pls_grid <- expand.grid(ncomp = 1:10)

set.seed(123)
fit_plsda <- train(
  x = x_train_only_boxcox,
  y = y_train,
  method = "pls",
  metric = "Kappa",
  tuneGrid = pls_grid,
  preProcess = c("center","scale"),
  trControl = ctrl_only_box
```

```
)

# Penalized
penal_grid <- expand.grid(
  alpha  = seq(0, 1, by = 0.25),
  lambda = 10^seq(-3, 1, length = 10)
)

set.seed(123)
fit_penal <- train(
  x = x_train_only_boxcox,
  y = y_train,
  method = "glmnet",
  metric = "Kappa",
  tuneGrid = penal_grid,
  preProcess = c("center","scale"),
  trControl = ctrl_only_box
)

# RDA
# Tuning grid for RDA
rda_grid <- expand.grid(
  gamma  = seq(0, 1, by = 0.25),
  lambda = seq(0, 1, by = 0.25)
)

set.seed(123)
fit_rda <- train(
```

```
  x = x_train_only_boxcox,
  y = y_train,
  method = "rda",
  metric = "Kappa",
  tuneGrid = rda_grid,
  preProcess = c("center","scale"),
  trControl = ctrl_only_box
)

# ONLY Removing Highly Correlated (Neural Network, Naive Bayes, FDA)
library(caret)
library(klaR)    # Naive Bayes
library(nnet)    # Neural Net
library(mda)     # for FDA (method = "fda")

set.seed(123)

ctrl_only_RHC <- trainControl(
  method = "cv",
  number = 10,
  classProbs = TRUE,
  summaryFunction = defaultSummary,   # Accuracy & Kappa
  savePredictions = "final"
)

# Neural Network
nn_grid <- expand.grid(
  size = c(1, 3, 5, 7, 9),
```

```
  decay = c(0, 0.001, 0.01, 0.1)
)

set.seed(123)
fit_nn <- train(
  x = x_train_only_RHC,
  y = y_train,
  method = "nnet",
  trace = FALSE,
  metric = "Kappa",
  tuneGrid = nn_grid,
  preProcess = c("center","scale","spatialSign"),
  trControl = ctrl_only_RHC
)

# NAive Bayes
nb_grid <- expand.grid(
  fL = c(0, 0.5, 1),
  usekernel = c(TRUE, FALSE),
  adjust = c(0.5, 1, 2)
)

set.seed(123)
fit_nb <- train(
  x = x_train_only_RHC,
  y = y_train,
  method = "nb",
  metric = "Kappa",
```

```
  tuneGrid = nb_grid,
  preProcess = c("center","scale"),
  trControl = ctrl_only_RHC
)

# FDA
fda_grid <- expand.grid(
  degree = 1:2,
  nprune = seq(5, 30, by = 5)
)

set.seed(123)
fit_fda <- train(
  x = x_train_only_RHC,
  y = y_train,
  method = "fda",
  metric = "Kappa",
  tuneGrid = fda_grid,
  preProcess = c("center","scale"),
  trControl = ctrl_only_RHC
)

# No boxcox no RHC (SVM,KNN)
library(caret)
library(kernlab)   # for svmRadial

set.seed(123)
```

```
ctrl_no_box_no_RHC <- trainControl(
  method = "cv",
  number = 10,
  classProbs = TRUE,
  summaryFunction = defaultSummary,   # Accuracy & Kappa
  savePredictions = "final"
)

# SVM
sigma_vals <- sigest(as.matrix(x_train_no_box_no_RHC))  # from kernlab, gives a range
svm_grid <- expand.grid(
  sigma = as.numeric(sigma_vals),
  C     = c(0.25, 0.5, 1, 2, 4)
 )

 set.seed(123)
 fit_svm <- train(
  x = x_train_no_box_no_RHC,
   y = y_train,
  method = "svmRadial",
  metric = "Kappa",
  tuneGrid = svm_grid,
  preProcess = c("center","scale"),
  trControl = ctrl_no_box_no_RHC
)

#KNN
knn_grid <- expand.grid(
```

```
  k = seq(1, 25, by = 2)   # odd ks from 1 to 25
 )

set.seed(123)
fit_knn <- train(
  x = x_train_no_box_no_RHC,
  y = y_train,
  method = "knn",
  metric = "Kappa",
  tuneGrid = knn_grid,
  preProcess = c("center","scale","spatialSign"),
  trControl = ctrl_no_box_no_RHC
)

# Tuning Plots
plot(fit_logit, main = "Logistic Regression – (No tuning grid)")
plot(fit_lda,   main = "LDA – (No tuning grid)")
plot(fit_qda,   main = "QDA – (No tuning grid)")
plot(fit_mda,   main = "MDA – Tuning (subclasses vs Kappa)")
plot(fit_plsda, main = "PLS-DA – Tuning (ncomp vs Kappa)")
plot(fit_penal, main = "Penalized (glmnet) – Tuning (alpha, lambda)")
plot(fit_rda,   main = "RDA – Tuning (gamma, lambda)")
plot(fit_nn,  main = "Neural Network – Tuning (size, decay)")
plot(fit_nb,  main = "Naive Bayes – Tuning (fL, usekernel, adjust)")
plot(fit_fda, main = "FDA – Tuning (degree, nprune)")
plot(fit_nn,  main = "Neural Network – Tuning (size, decay)")
plot(fit_nb,  main = "Naive Bayes – Tuning (fL, usekernel, adjust)")
plot(fit_fda, main = "FDA – Tuning (degree, nprune)")
```

```
# SUmmary Table
##################################################################
### MASTER SUMMARY TABLE FOR ALL MODELS (TRAINING PERFORMANCE)
##################################################################

all_models <- list(
  Logistic     = fit_logit,
  LDA          = fit_lda,
  QDA          = fit_qda,
  MDA          = fit_mda,
  PLS_DA       = fit_plsda,
  Penalized    = fit_penal,
  RDA          = fit_rda,
  NeuralNet    = fit_nn,
  NaiveBayes   = fit_nb,
  FDA          = fit_fda,
  SVM_Radial   = fit_svm,
  KNN          = fit_knn
)

# ---- 1. Extract max Kappa for each model ----
train_kappa_all <- sapply(all_models, function(m) {
  max(m$results$Kappa, na.rm = TRUE)
})

# ---- 2. Extract max Accuracy ----
train_acc_all <- sapply(all_models, function(m) {
```

```
  max(m$results$Accuracy, na.rm = TRUE)
})

# ---- 3. Extract BEST tuning parameter(s) as text ----
best_tune_all <- sapply(all_models, function(m) {
  if (!is.null(m$bestTune) && ncol(m$bestTune) > 0) {
    paste0(
      apply(m$bestTune, 1, function(row)
        paste(names(row), "=", row, collapse = ", ")
      ),
      collapse = "; "
    )
  } else {
    "None"
  }
})

# ---- 4. Build final summary table ----
all_model_summary <- data.frame(
  Model           = names(train_kappa_all),
  Training_Kappa    = as.numeric(train_kappa_all),
  Training_Accuracy  = as.numeric(train_acc_all),
  Best_Tuning       = best_tune_all,
  row.names = NULL
)

# ---- 5. Sort by Training Kappa (descending) ----
```

```
all_model_summary <- all_model_summary[order(-
all_model_summary$Training_Kappa), ]

all_model_summary
```

```
#### Best models ####

# NAIVE BAYES

pred_nb <- predict(fit_nb, newdata = x_test_only_RHC)

# Confusion Matrix

cm_nb <- confusionMatrix(pred_nb, Y_test)

cm_nb

# Extract metrics

nb_accuracy <- cm_nb$overall['Accuracy']

nb_kappa    <- cm_nb$overall['Kappa']

nb_accuracy

nb_kappa

# QDA

pred_qda <- predict(fit_qda, newdata = x_test_both_box_RHC)

# Confusion Matrix
```

```
cm_qda <- confusionMatrix(pred_qda, Y_test)
cm_qda

# Extract metrics
qda_accuracy <- cm_qda$overall['Accuracy']
qda_kappa    <- cm_qda$overall['Kappa']

qda_accuracy
qda_kappa
```